

Uni-App 封装request请求的传参详解

在使用封装后的request网络请求时，传参是连接前端与后端的关键环节。不同请求方式（GET/POST/PUT等）、不同数据类型（简单数据/复杂对象/文件）的传参规则存在差异，若使用不当会导致接口请求失败或数据异常。本文基于此前封装的Promise请求工具，系统讲解各类场景下的传参方法、注意事项及优化技巧，确保参数能准确、高效地传递至后端。

一、传参核心基础：请求配置中的参数载体

在封装的request工具中，所有请求参数均通过 `options` 配置对象传递，核心关联三个属性：`method`（请求方式）、`data`（参数数据）、`header`（请求头，影响参数解析方式）。不同请求方式下，`data` 的处理逻辑和传递位置完全不同，这是传参的核心差异点。

1. 核心配置与参数的关联逻辑

Code block

```
1  // 回顾封装的request工具核心逻辑（参数处理部分）
2  export const request = (options) => {
3    const requestOptions = {
4      url: BASE_CONFIG.baseUrl + options.url,
5      method: options.method?.toUpperCase() || 'GET', // 默认为GET
6      data: options.data || {}, // 核心参数载体
7      header: { ...BASE_CONFIG.defaultHeader, ...options.header }, // 影响参数解析
8      timeout: BASE_CONFIG.timeout
9    };
10
11    // 关键：GET与非GET请求的参数处理差异（封装内已实现）
12    if (requestOptions.method === 'GET') {
13      // GET请求：data会自动拼接到URL上（如?name=test&age=18）
14      requestOptions.url = 拼接参数(requestOptions.url, requestOptions.data);
15      requestOptions.data = {}; // GET请求data字段无效，清空
16    }
17    // 非GET请求（POST/PUT等）：data作为请求体传递
18
19    return new Promise((resolve, reject) => {
20      uni.request({...requestOptions, success, fail});
21    });
22  };
```



封装工具已内置GET请求参数拼接逻辑，开发者无需手动处理URL拼接，只需统一将参数放入 `data` 中即可，极大降低了传参复杂度。

2. 传参的核心原则

- 1. 匹配请求方式：** GET请求参数通过URL传递，POST/PUT等通过请求体传递，需根据后端接口要求选择正确方式；
- 2. 统一数据格式：** 简单数据（字符串/数字）直接传递，复杂数据（对象/数组）需确保后端能解析（如JSON格式）；
- 3. 适配请求头：** `Content-Type` 决定后端解析参数的方式，需与参数格式匹配（如表单提交用 `application/x-www-form-urlencoded`）；
- 4. 避免敏感信息：** 密码、Token等敏感信息禁止通过GET请求的URL传递，需用POST请求体或请求头传递。

二、不同请求方式的传参方法（实战示例）

结合封装的 `get`、`post` 等简化方法，针对不同请求方式给出具体传参示例，覆盖90%以上的开发场景。

1. GET请求传参：简单数据与URL拼接

GET请求适用于获取数据，参数通常为简单类型（字符串、数字、布尔值），封装工具会自动将 `data` 中的参数拼接到URL末尾。

(1) 基础传参：单个/多个简单参数

Code block

```
1  // 1. 导入简化方法
2  import { get } from '@utils/request';
3
4  // 2. 接口调用：获取商品列表（参数：分类ID、页码、每页条数）
5  async getGoodsList() {
6    try {
7      // 参数放入data中，工具自动拼接为：/goods/list?categoryId=1&page=1&pageSize=10
8      const data = await get('/goods/list', {
9        categoryId: 1, // 数字类型
10       page: 1,
11       pageSize: 10,
12       sort: 'price_asc' // 字符串类型
13     });
14     console.log('商品列表: ', data);
15   } catch (error) {
```

```
16     console.error('请求失败: ', error);
17   }
18 }
```

(2) 特殊场景：数组参数传递

GET请求传递数组时，后端通常支持两种格式：`ids=1&ids=2&ids=3`（多值）或 `ids=[1,2,3]`（JSON格式），需与后端协商一致。

Code block

```
1  // 方式1: 多值格式 (推荐, 兼容性更好)
2  async getSelectedGoods() {
3    const ids = [1, 2, 3];
4    // 手动处理为多值参数: /goods/selected?ids=1&ids=2&ids=3
5    const params = ids.reduce((acc, id, index) => {
6      acc[`ids[${index}]`] = id;
7      return acc;
8    }, {});
9    const data = await get('/goods/selected', params);
10 }
11
12 // 方式2: JSON格式 (需后端支持解析URL中的JSON)
13 async getSelectedGoods() {
14   const ids = [1, 2, 3];
15   // 传递为: /goods/selected?ids=%5B1%2C2%2C3%5D (JSON转义后)
16   const data = await get('/goods/selected', {
17     ids: JSON.stringify(ids) // 转为JSON字符串
18   });
19 }
```

2. POST请求传参：请求体传参的多种格式

POST请求适用于提交数据，参数通过请求体传递，支持多种数据格式，核心通过 `Content-Type` 请求头控制，封装工具默认使用 `application/json`。

(1) JSON格式传参（默认，推荐）

适用于传递复杂数据（对象、数组），后端需支持JSON解析，是前后端交互的主流方式。

Code block

```
1  import { post } from '@utils/request';
2
3  // 提交商品信息 (复杂对象参数)
4  async addGoods() {
```

```

5   try {
6     const goodsData = {
7       name: '华为Mate 60 Pro', // 字符串
8       price: 6999, // 数字
9       stock: 100,
10      specs: [ // 数组类型
11        { color: '黑色', memory: '12+256G' },
12        { color: '白色', memory: '12+512G' }
13      ],
14      isOnSale: true // 布尔值
15    };
16    // 工具默认设置Content-Type: application/json, 参数作为请求体传递
17    const result = await post('/goods/add', goodsData);
18    console.log('商品新增成功: ', result);
19  } catch (error) {
20    console.error('新增失败: ', error);
21  }
22 }

```

(2) 表单格式传参 (application/x-www-form-urlencoded)

适用于传递简单键值对，常见于传统后端接口或第三方接口（如支付接口），需手动修改请求头。

Code block

```

1  // 登录接口 (表单格式传参)
2  async userLogin() {
3    try {
4      const formData = {
5        username: 'test_user',
6        password: '123456a'
7      };
8      // 1. 修改请求头为表单格式
9      // 2. 工具会将data转为: username=test_user&password=123456a 放入请求体
10     const result = await post('/user/login', formData, {
11       header: {
12         'Content-Type': 'application/x-www-form-urlencoded'
13       }
14     });
15     // 存储Token
16     uni.setStorageSync('token', result.token);
17   } catch (error) {
18     console.error('登录失败: ', error);
19   }
20 }

```

(3) 文件上传 (multipart/form-data)

上传图片、视频等文件时，需使用 `multipart/form-data` 格式，Uni-App提供 `uni.uploadFile` 接口，可基于此扩展封装工具。

Code block

```
1  // 1. 扩展request工具，添加文件上传方法
2  export const uploadFile = (options) => {
3    const { url, filePaths, formData = {}, loadingText = '上传中...' } = options;
4    if (!filePaths || filePaths.length === 0) {
5      return Promise.reject(new Error('请选择需要上传的文件'));
6    }
7
8    uni.showLoading({ title: loadingText, mask: true });
9
10   return new Promise((resolve, reject) => {
11     uni.uploadFile({
12       url: BASE_CONFIG.baseUrl + url,
13       filePaths, // 文件路径数组（支持多文件）
14       formData, // 额外表单参数
15       header: {
16         ...BASE_CONFIG.defaultHeader,
17         'Content-Type': 'multipart/form-data',
18         Authorization: `Bearer ${uni.getStorageSync('token')}` // 携带Token
19       },
20       success: (res) => {
21         uni.hideLoading();
22         const data = JSON.parse(res.data); // 解析JSON响应
23         if (data.code === 200) {
24           resolve(data.data);
25         } else {
26           uni.showToast({ title: data.msg, icon: 'none' });
27           reject(new Error(data.msg));
28         }
29       },
30       fail: (error) => {
31         uni.hideLoading();
32         uni.showToast({ title: '文件上传失败', icon: 'none' });
33         reject(error);
34       }
35     });
36   });
37 };
38
39 // 2. 页面中使用：上传商品图片
40 async uploadGoodsImage() {
```

```
41 // 选择图片
42 const chooseRes = await uni.chooseImage({ count: 1 });
43 // 上传图片
44 const uploadRes = await uploadFile({
45   url: '/goods/uploadImage',
46   filePaths: chooseRes.tempFilePaths,
47   formData: { goodsId: 123 }, // 额外参数 (关联商品ID)
48   loadingText: '图片上传中...'
49 });
50 console.log('图片地址: ', uploadRes.imageUrl);
51 }
```

3. PUT/DELETE请求传参：按需选择传递方式

PUT（更新数据）、DELETE（删除数据）请求的传参逻辑与POST类似，可根据后端要求选择“请求体传参”或“URL路径传参”。

Code block

```
1 import { put, del } from '@utils/request';
2
3 // 1. PUT请求：更新商品信息（请求体传参，同POST）
4 async updateGoods() {
5   const updateData = {
6     id: 123, // 商品ID
7     name: '华为Mate 60 Pro 新品',
8     price: 7199
9   };
10   const result = await put('/goods/update', updateData);
11 }
12
13 // 2. DELETE请求：删除商品（URL路径传参，更直观）
14 async deleteGoods(goodsId) {
15   // 方式1: URL路径传参（推荐，符合RESTful规范）
16   const result = await del(`/goods/delete/${goodsId}`);
17
18   // 方式2: 请求体传参（适用于批量删除）
19   // const result = await del('/goods/batchDelete', { ids: [123, 456] });
20 }
```

三、复杂场景传参技巧

针对特殊业务场景（如携带Token、传递日期、加密参数），给出传参解决方案，确保参数传递的安全性和准确性。

1. 携带身份凭证（Token/Authorization）

用户登录后，需通过请求头携带Token实现身份验证，推荐在请求拦截器中统一处理，无需每次传参时手动添加。

Code block

```
1  // 1. 在request工具的请求拦截器中添加Token
2  const INTERCEPTORS = {
3    requestInterceptor: (config) => {
4      const token = uni.getStorageSync('token');
5      if (token) {
6        // 后端约定的Token格式（常见：Bearer + 空格 + Token）
7        config.header.Authorization = `Bearer ${token}`;
8      }
9      return config;
10   }
11 };
12
13 // 2. 页面中调用接口时无需手动传Token
14 async getUserInfo() {
15   // 工具会自动在请求头中添加Authorization
16   const userInfo = await get('/user/info');
17 }
```

2. 日期类型参数传递

日期类型直接传递会导致格式混乱，需统一转为“时间戳”或“标准ISO格式”，前后端约定一致即可。

Code block

```
1  // 传递日期参数：查询近7天的订单
2  async getRecentOrders() {
3    const today = new Date();
4    const sevenDaysAgo = new Date(today.getTime() - 7 * 24 * 60 * 60 * 1000);
5
6    // 方式1：传递时间戳（推荐，无格式问题）
7    const data1 = await get('/order/list', {
8      startDate: sevenDaysAgo.getTime(), // 1699996800000
9      endDate: today.getTime()
10   });
11
12   // 方式2：传递ISO格式字符串（后端需支持解析）
13   const data2 = await get('/order/list', {
14     startDate: sevenDaysAgo.toISOString(), // "2023-11-14T00:00:00.000Z"
```

```
15     endDate: today.toISOString()
16   });
17 }
```

3. 加密参数传递

敏感数据（如支付信息、身份证号）需加密后传递，确保数据安全，加密逻辑可在请求拦截器中统一处理。

Code block

```
1  // 1. 导入加密工具（如AES加密）
2  import { aesEncrypt } from '@utils/encrypt';
3
4  // 2. 请求拦截器中加密敏感参数
5  const INTERCEPTORS = {
6    requestInterceptor: (config) => {
7      // 仅对支付接口的参数加密
8      if (config.url.includes('/payment')) {
9        // 加密请求体数据
10       config.data = {
11         encryptData: aesEncrypt(JSON.stringify(config.data), 'secretKey')
12       };
13       // 告知后端数据已加密
14       config.header['Encrypt-Type'] = 'AES';
15     }
16     return config;
17   }
18 };
19
20 // 3. 页面中正常传参，工具自动加密
21 async submitPayment() {
22   const payData = {
23     orderId: 'ORD123456',
24     amount: 999,
25     bankCard: '622848*****1234' // 敏感信息
26   };
27   const result = await post('/payment/submit', payData);
28 }
```

4. 自定义请求头传参

部分接口需通过请求头传递特殊参数（如设备信息、接口版本），可在调用请求方法时通过 `options.header` 传入。


```
1 // 传递设备信息和接口版本
2 async getSystemConfig() {
3   // 获取设备信息
4   const systemInfo = uni.getSystemInfoSync();
5   const result = await get('/system/config', {}, {
6     header: {
7       'Device-Type': systemInfo.platform, // 设备类型 (ios/android)
8       'Api-Version': 'v2.0' // 接口版本
9     }
10  });
11 }
```

四、传参常见问题与解决方案

整理传参过程中高频出现的问题，分析原因并给出具体解决方法，避免踩坑。

1. 问题1：GET请求参数传递后，后端接收不到

常见原因：参数格式错误（如数组未处理）、URL编码问题、参数名与后端不一致。

解决方案：

1. 检查参数名是否与后端接口文档完全一致（区分大小写）；
2. 数组参数需按后端要求处理为多值格式或JSON字符串；
3. 特殊字符（如中文、空格）需手动编码（工具已内置 `encodeURIComponent`，若自定义拼接需注意）：

```
// 手动处理特殊字符（工具已内置，无需重复处理）
const keyword = '华为 手机';
const data = await get('/goods/search', {
  keyword: encodeURIComponent(keyword) // 编码为"华为%20手机"
});
```

2. 问题2：POST请求参数后端解析为null

常见原因： `Content-Type` 与参数格式不匹配、参数未序列化。

解决方案：

1. 若传递JSON对象，确保请求头为 `application/json`（工具默认）；
2. 若传递表单数据，需将请求头改为 `application/x-www-form-urlencoded`，且参数为简单键值对；
3. 避免传递循环引用的对象（会导致JSON序列化失败）。

3. 问题3：文件上传时，后端接收不到文件或参数

常见原因：文件路径错误、`Content-Type` 未设置为 `multipart/form-data`、额外参数未放入 `formData`。

解决方案：

1. 确保 `filePaths` 使用 `uni.chooseImage` 返回的 `tempFilePaths`；
2. 文件上传必须使用 `uni.uploadFile` 接口，而非普通 `uni.request`；
3. 额外参数需放入 `formData` 中，而非 `data`。

4. 问题4：Token传递后，仍提示“未登录”

常见原因：Token格式错误、Token过期、请求头键名与后端不一致。

解决方案：

1. 检查Token格式是否符合后端要求（如是否需要添加 `Bearer` 前缀）；
2. 通过控制台Network面板查看请求头，确认Authorization是否正确携带；
3. 若Token过期，需在响应拦截器中处理重新登录逻辑。

5. 问题5：复杂对象参数传递后，后端接收的数据不完整

常见原因：对象中包含 `undefined` 或 `function` 属性、嵌套层级过深。

解决方案：

1. 删除参数中的 `undefined`、`null` 或 `function` 属性（可通过工具函数过滤）；

```
// 过滤无效属性的工具函数
const filterParams = (obj) => {
  return Object.fromEntries(
    Object.entries(obj).filter(([_, value]) => value !== undefined
    && value !== null)
  );
};

// 调用前过滤参数
const goodsData = filterParams({
  name: '手机',
  price: 5000,
  desc: undefined // 会被过滤
});
```

2. 若后端不支持深层嵌套对象，可将其扁平化或转为JSON字符串传递。

五、传参最佳实践总结

结合前文内容，提炼request请求传参的最佳实践，形成规范的传参流程，提升开发效率和接口稳定性。

1. **接口分层封装**：将所有接口请求统一放在 `src/api` 目录下，参数定义与后端接口文档对齐，页面仅调用API方法，不直接写请求逻辑；
2. **遵循RESTful规范**：GET（查询）、POST（新增）、PUT（更新）、DELETE（删除），URL路径传参用于标识资源（如 `/goods/123`）；
3. **统一参数处理**：在请求拦截器中完成Token添加、参数加密、格式转换等通用逻辑，减少重复代码；
4. **参数校验前置**：页面调用API前，对必填参数进行校验（如是否为空、格式是否正确），避免无效请求；

```
// 前置参数校验
async submitOrder(params) {
  // 校验必填参数
  if (!params.goodsId) throw new Error('商品ID不能为空');
  if (params.num < 1) throw new Error('购买数量不能小于1');
  // 调用接口
  return await post('/order/submit', params);
}
```

5. **善用工具调试**：通过HBuilderX的“网络请求”面板或浏览器F12的Network面板，查看请求URL、请求头、请求体是否与预期一致，快速定位问题。

（注：文档部分内容可能由 AI 生成）